

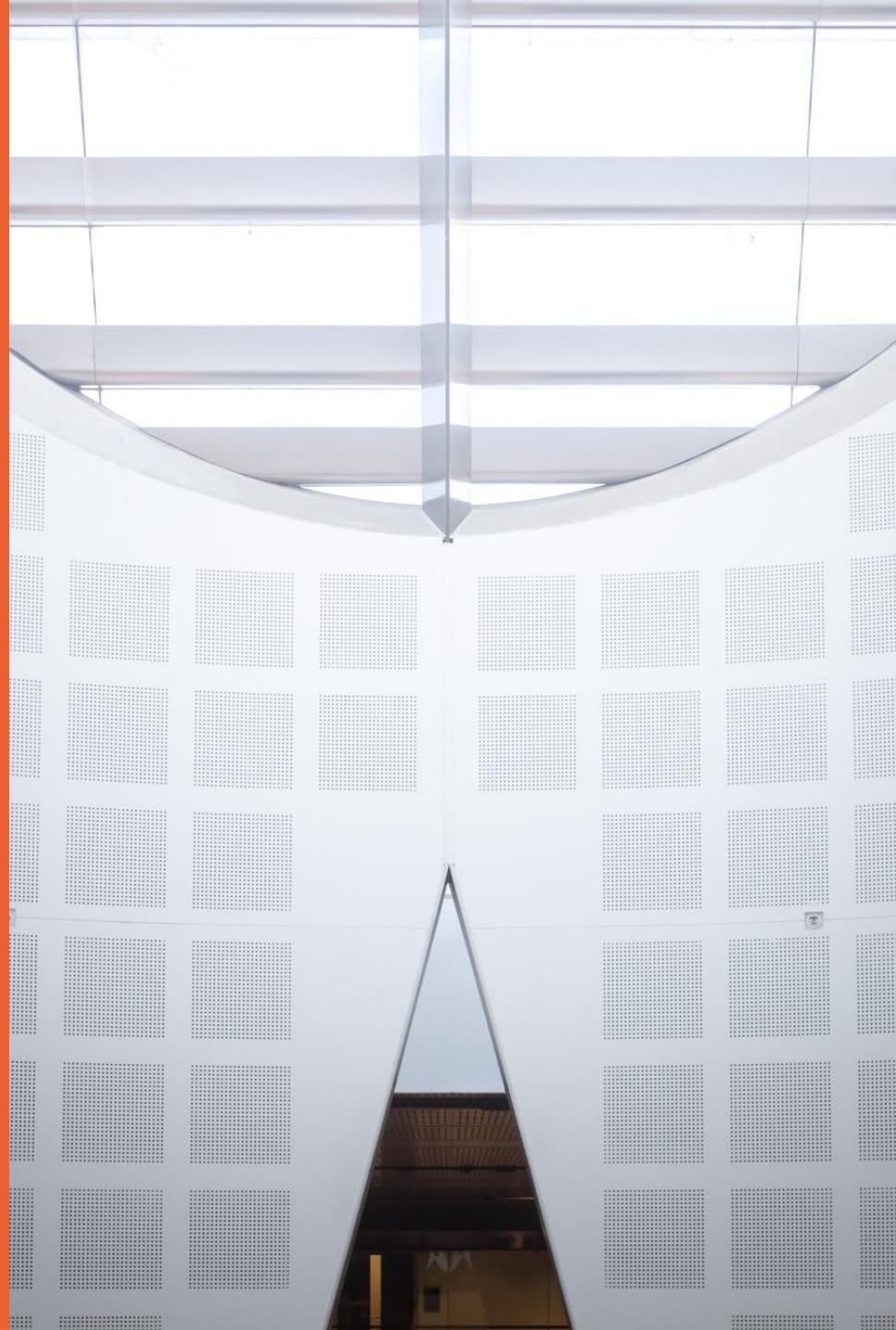
Graph Convolutional Networks

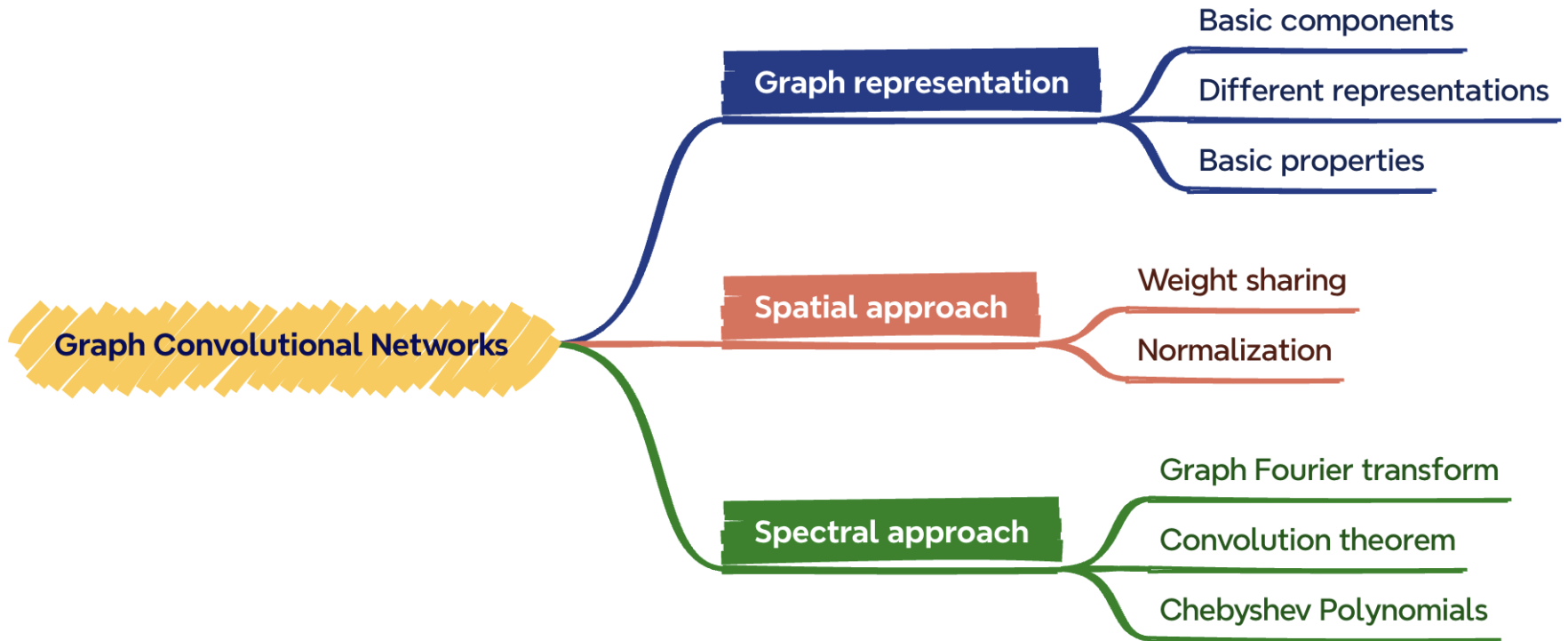
Qianyang Chen

School of Computer Science



THE UNIVERSITY OF
SYDNEY





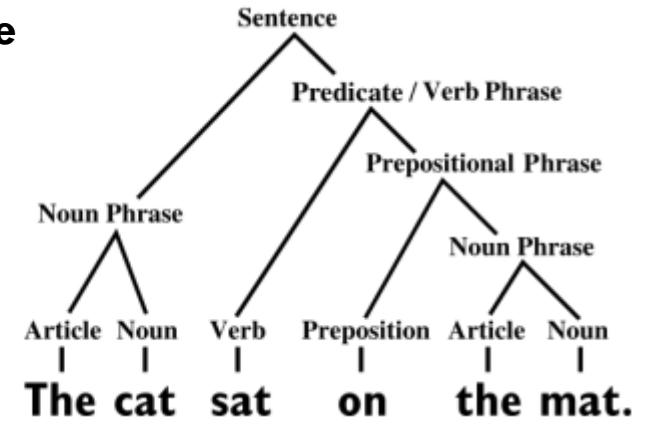
Background: Grid structured data

IMAGENET

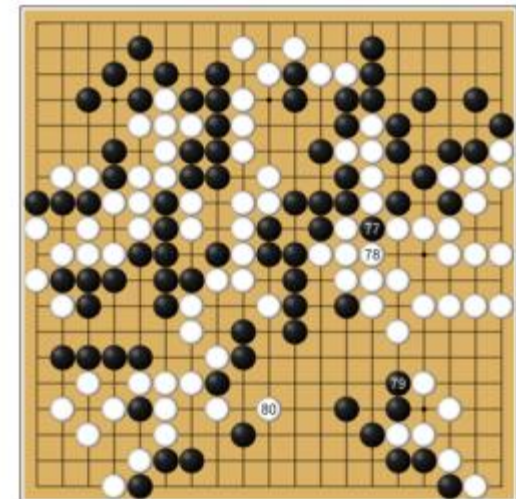


Natural language
processing (NLP)

...



Grid games



Speech data



Background: Graph data

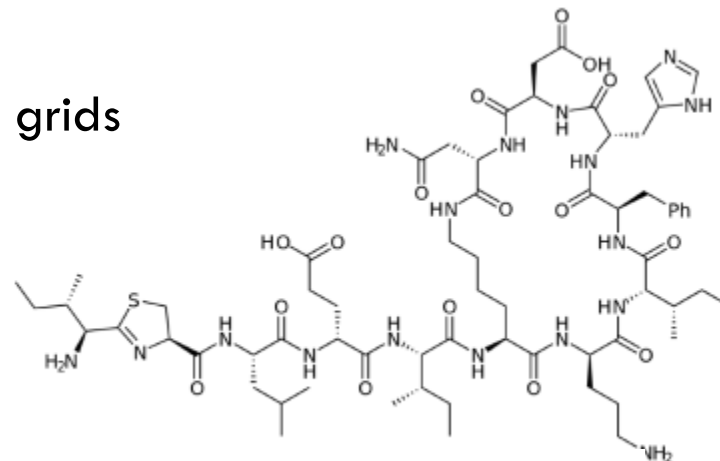
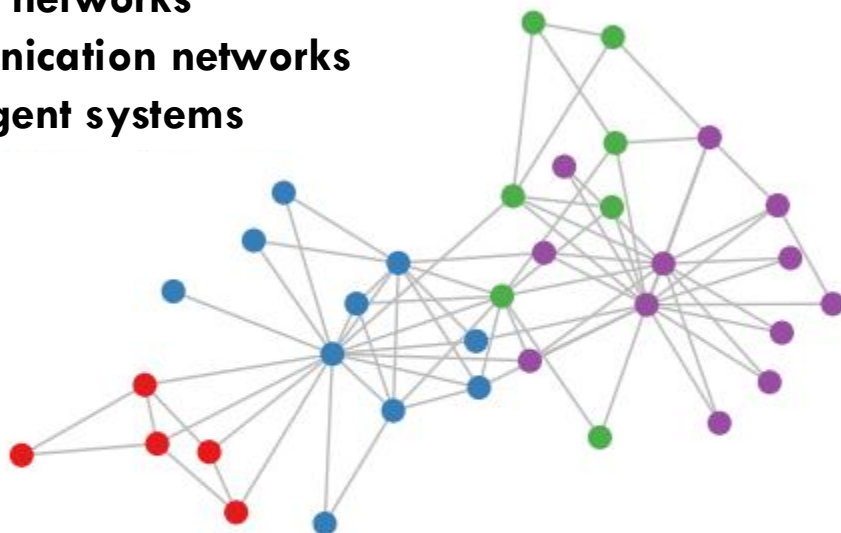
A lot of real-world data does not 'live' on grids

Social networks

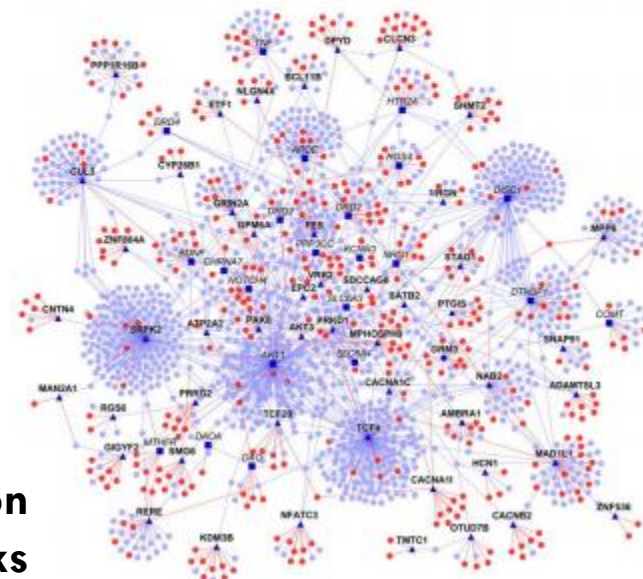
Citation networks

Communication networks

Multi-agent systems



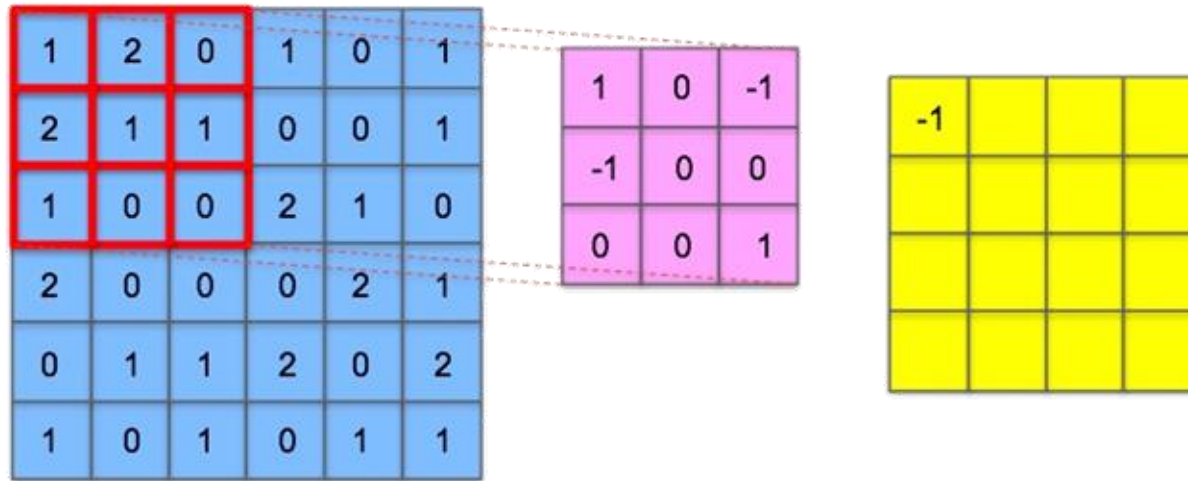
Molecules



Protein interaction networks

Standard deep learning architectures like CNNs and RNNs don't work here!

Background: Difficulties on graph data processing



Fixed number of neighboring pixels

&

Implicit spatial order of neighboring pixels

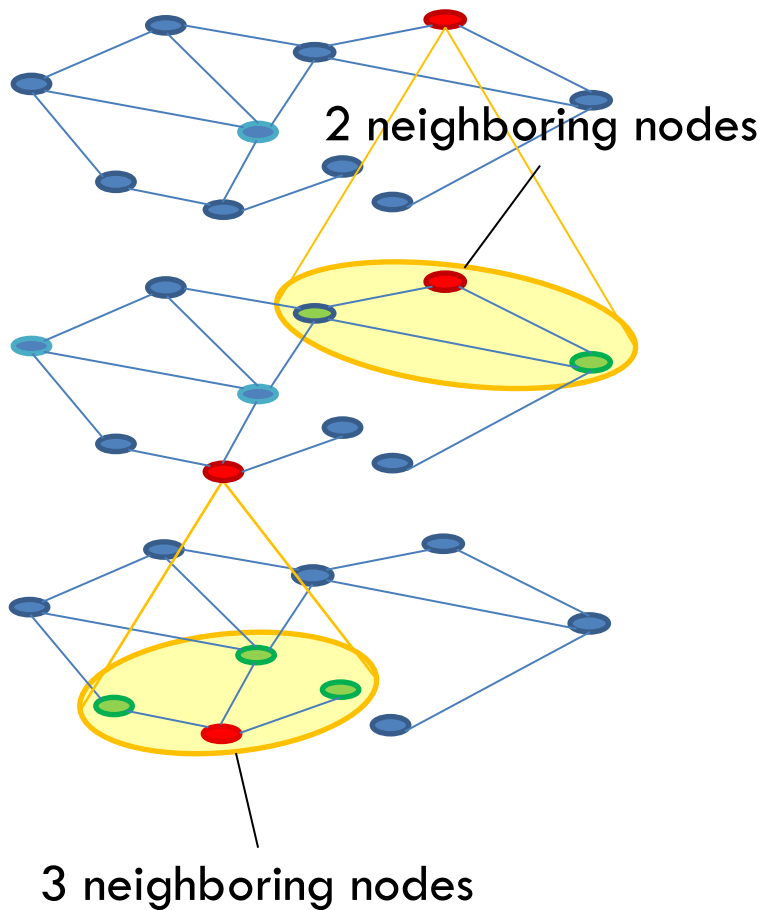


Kernel with fixed size

&

Weights with implicit order

Background: Difficulties on graph data processing



Varying number of neighboring nodes

&

No implicit spatial order of neighboring nodes



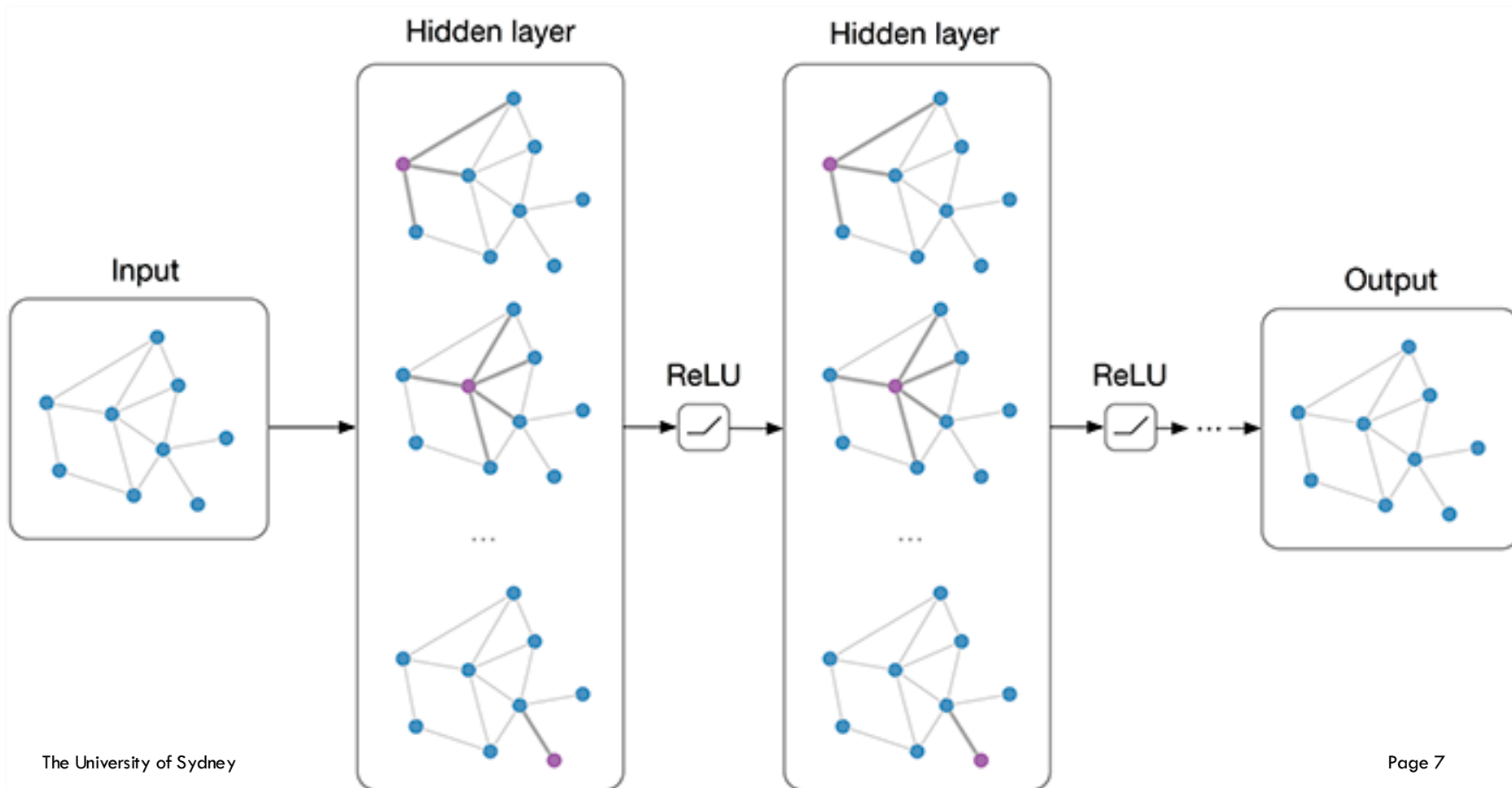
Kernel with varying size

&

Weights with undecided order

Graph Convolutional Networks (GCN)

Extract features from the graph data.



Preliminaries: Representing graphs

Graphs consist of

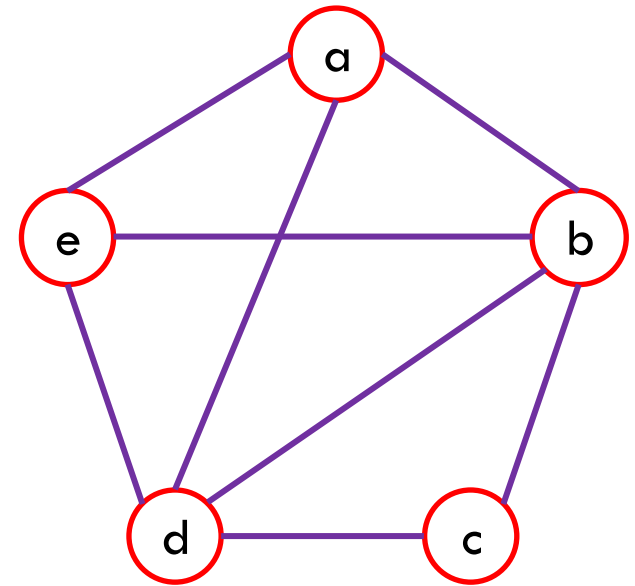
- ◆ Nodes (or vertices)
- ◆ Edges connecting pairs of nodes

Formally

$$G = (V, E)$$

- ◆ $V = \{v_i | i = 1 \dots N\}$: Node set, containing $|V| = N$ nodes

- ◆ $E = \{e_{ij} | v_i \text{ is connected to } v_j\}$: Edge set



Preliminaries: Representing graphs

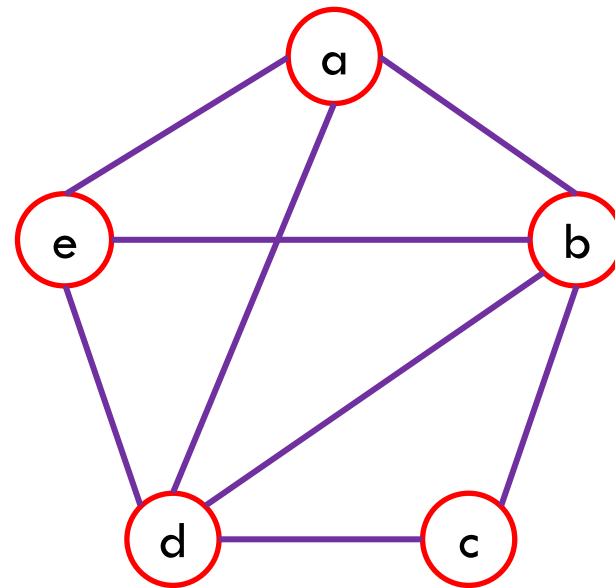
Edge representation

- ◆ Edge list
- ◆ Adjacency matrix
- ◆ Incidence matrix

1. **Edge list:** list of all edges

Edges:

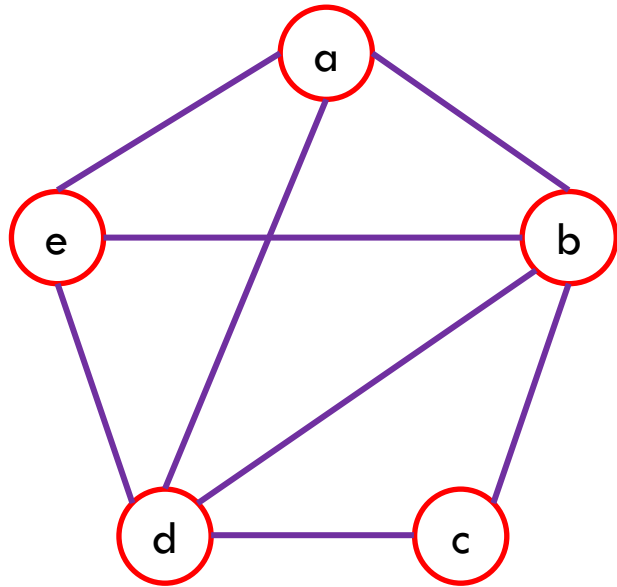
$(a,b), (a,d), (a,e), (b,c), (b,d), (b,e)$
 $, (c,d), (d,e)$



G

Preliminaries: Representing graphs

2. Adjacency matrix



G

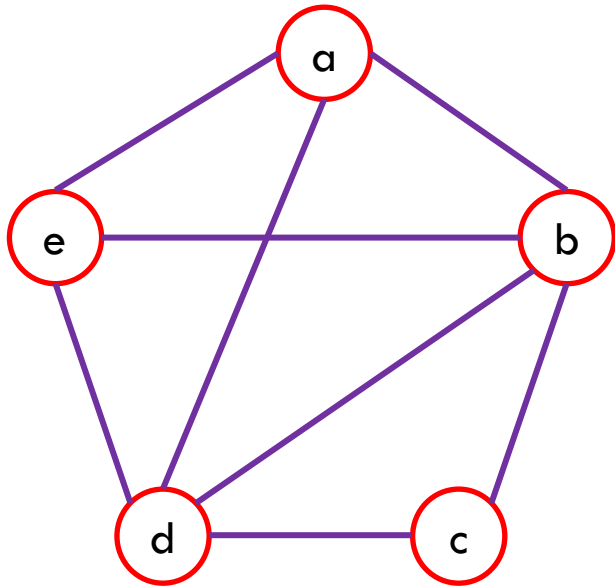
$A =$

$$\begin{matrix} & a & b & c & d & e \\ \begin{matrix} a \\ b \\ c \\ d \\ e \end{matrix} & \begin{pmatrix} 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 & 1 \\ 1 & 1 & 0 & 1 & 0 \end{pmatrix} \end{matrix}$$

$N = 5$

Preliminaries: Representing graphs

2. Adjacency matrix with self connections



G

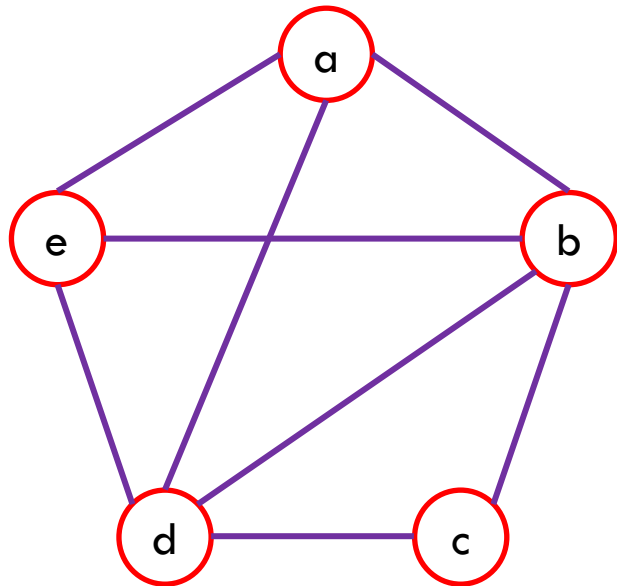
$$\hat{A} =$$

$$N = 5$$

	a	b	c	d	e
a	1	1	0	1	1
b	1	1	1	1	1
c	0	1	1	1	0
d	1	1	1	1	1
e	1	1	0	1	1

Preliminaries: Representing graphs

2. Adjacency matrix (weighted graph)



G

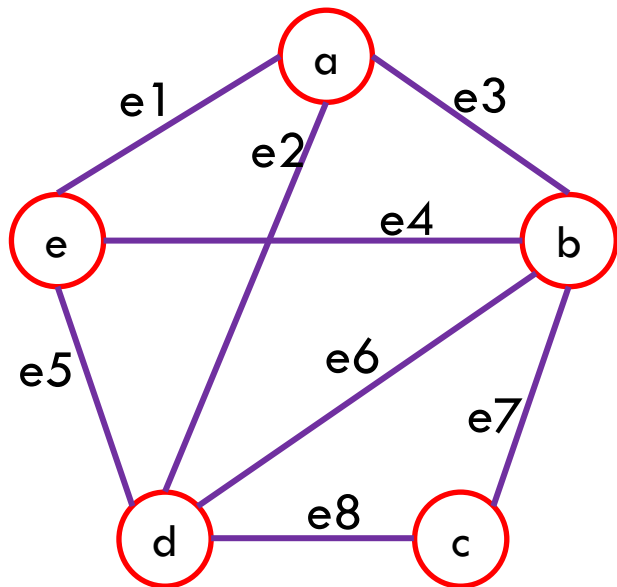
$$A_W =$$

$$N = 5$$

$$\begin{matrix} & \begin{matrix} a & b & c & d & e \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \\ e \end{matrix} & \begin{pmatrix} w_{aa} & w_{ab} & 0 & w_{ad} & w_{ae} \\ w_{ba} & w_{bb} & w_{bc} & w_{bd} & w_{be} \\ 0 & w_{cb} & w_{cc} & w_{cd} & 0 \\ w_{da} & w_{db} & w_{dc} & w_{dd} & w_{de} \\ w_{ea} & w_{eb} & 0 & w_{ed} & w_{ee} \end{pmatrix} \end{matrix}$$

Preliminaries: Representing graphs

3. Incidence matrix



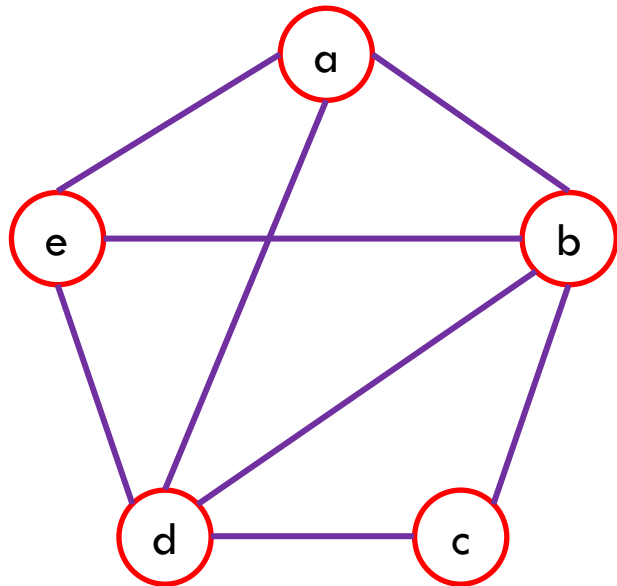
G

$$D = \begin{matrix} & \begin{matrix} e1 & e2 & e3 & e4 & e5 & e6 & e7 & e8 \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \\ e \end{matrix} & \begin{pmatrix} 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 & 1 & 0 & 0 & 0 \end{pmatrix} \end{matrix}$$

$$N = 5$$

Preliminaries: Representing graphs

Degree matrix: Denoting degree of each node



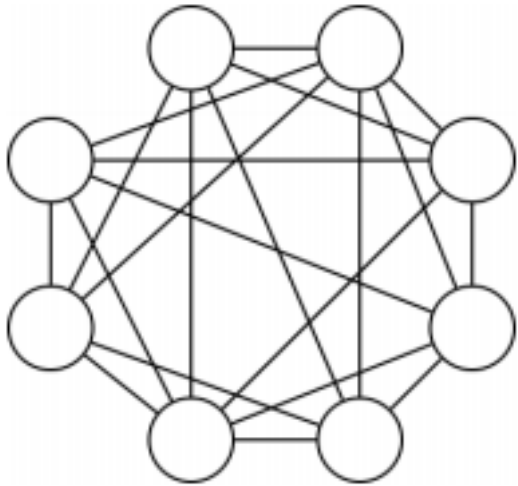
G

N = 5

$$D = \begin{matrix} & \begin{matrix} a & b & c & d & e \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \\ e \end{matrix} & \begin{pmatrix} 3 & 0 & 0 & 0 & 0 \\ 0 & 4 & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 & 0 \\ 0 & 0 & 0 & 4 & 0 \\ 0 & 0 & 0 & 0 & 3 \end{pmatrix} \end{matrix}$$

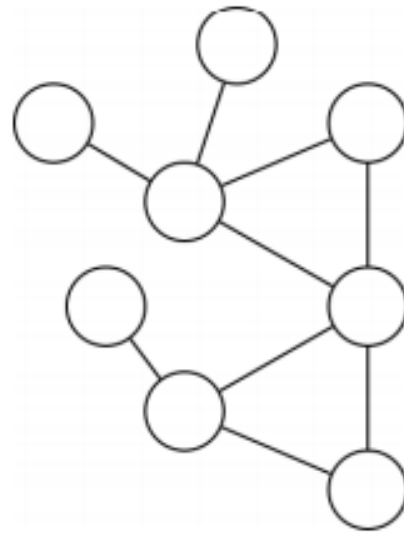
Preliminaries: Basic properties

Dense graph: $|E| \approx |V|^2$
Or $|E| = O(|V|^2)$



A large fraction of pairs of nodes are connected by edges

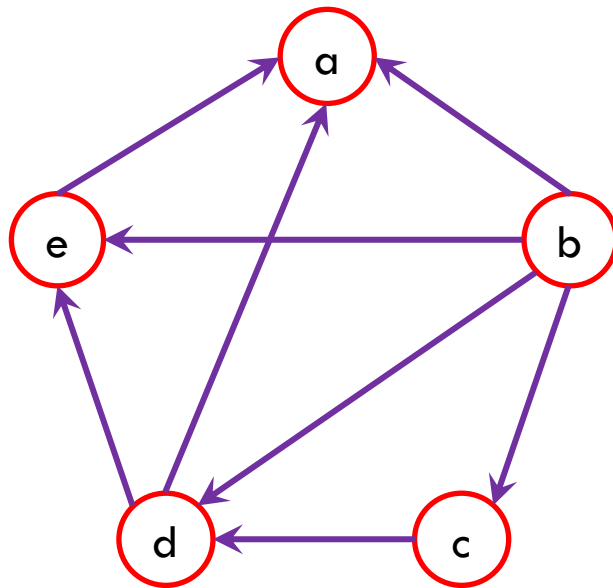
Sparse graph: $|E| \approx |V|$
Or $|E| = O(|V|)$



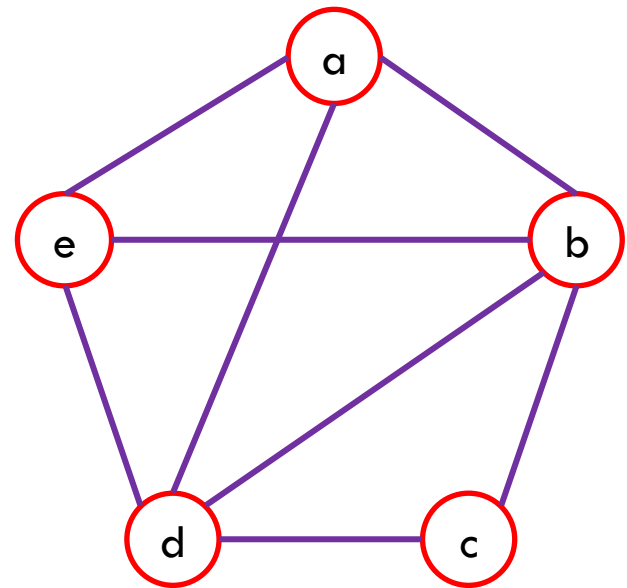
Each node has only a few edges

Preliminaries: Basic properties

Directed graph: Each edge has a direction

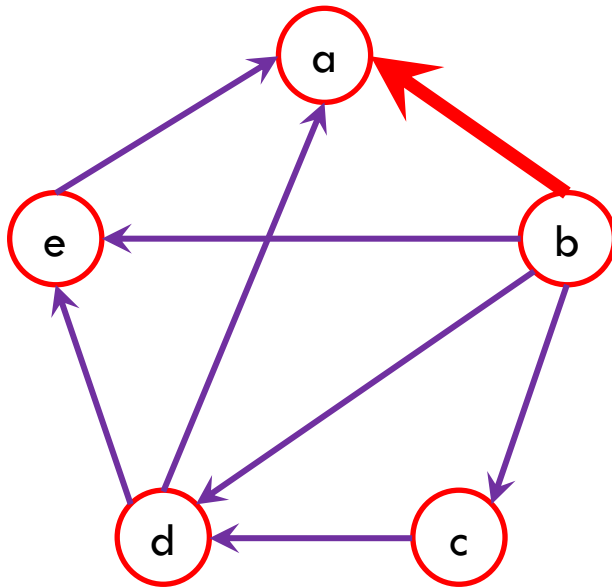


Undirected graph: No direction is associated with edges



Preliminaries: Representing graphs

Adjacency matrix of directed graph could be **asymmetric**



$A =$

	a	b	c	d	e
a	0	0	0	0	0
b	1	0	1	1	1
c	0	0	0	1	0
d	1	0	0	0	1
e	1	0	0	0	0

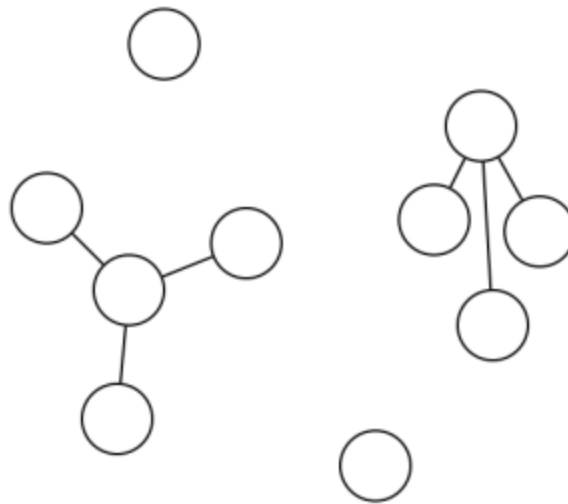
$N = 5$

Preliminaries: Basic properties

Connected component: A subgraph in which any two nodes are connected to each other by paths.

Theorem: The nodes of a graph G can be partitioned into connected components so that a node v is reachable from w if and only if they are in the same connected components

4 connected components exist in the graph below



Preliminaries: Representing graphs

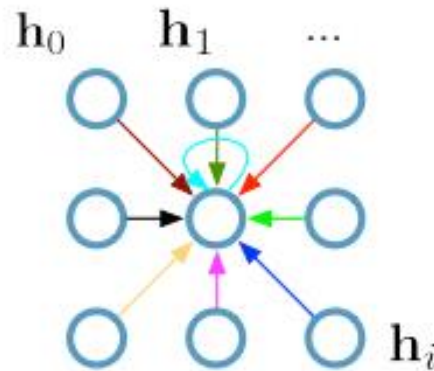
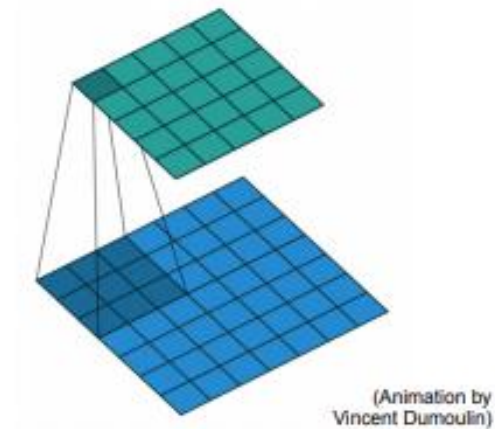
List of basics notations

- $G = (V, E)$
- $V = \{v_i | i = 1 \dots N\}$: Node set, containing $|V| = N$ nodes
- $E = \{e_{ij} | v_i \text{ is connected to } v_j\}$: Edge set
- $X \in R^{N \times d}$: Node attribute matrix
- Adjacency matrix: $A \in R^{N \times N}$, $A_{ij} \in \{0, 1\}$, denoting the existence of e_{ij}
- I_N : Identity matrix, denoting self-connections
- Adjacency matrix with self-connections: $\hat{A} = A + I_N$
- Degree: number of edges connected to a node
- Degree matrix: $D \in R^{N \times N}$ (computed from A), diagonal matrix denoting the degree of each node. And $\hat{D} \in R^{N \times N}$ is computed from $\hat{A}$

Spatial approach

Convolution in CNN

Single CNN layer with 3x3 filter:



Update for a single pixel:

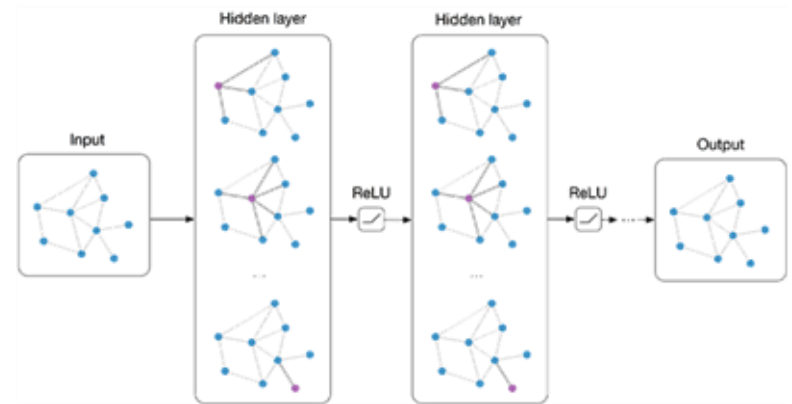
- Transform messages individually $W_i h_i$
- Add everything up $\sum_i W_i h_i$

$h_i \in R^F$ are (hidden layer) activations of a pixel/node

Full update:

$$h_4^{(l+1)} = \sigma(W_0^{(l)} h_0^{(l)} + W_1^{(l)} h_1^{(l)} + \dots + W_8^{(l)} h_8^{(l)})$$

Convolution in GCN



$$H^{(l+1)} = \sigma(\hat{D}^{-\frac{1}{2}} \hat{A} \hat{D}^{-\frac{1}{2}} H^l W^l)$$

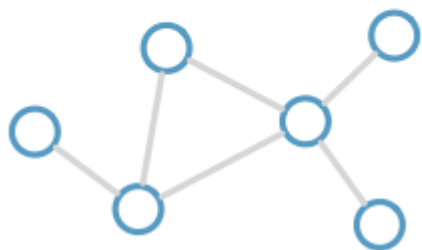
- Adjacency matrix with self-connections: $\hat{A} = A + I_N$
- Degree matrix: $\hat{D} \in R^{N \times N}$ (computed from $\hat{A}$). Note: degree = number of edges connected to a node.
- Hidden layers: H^l is input, H^{l+1} is output
- Filter parameters matrix: $W^l \in \mathbb{R}^{C \times F}$, where C = number of input channels, F = number of filters
- Activation function: $\sigma(\cdot)$ to introduce non-linearity

Why

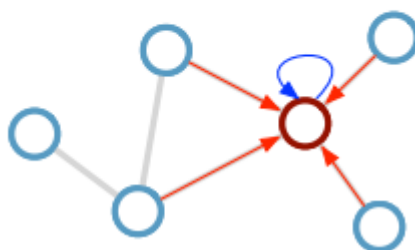
$$H^{(l+1)} = \sigma(\hat{D}^{-\frac{1}{2}} \hat{A} \hat{D}^{-\frac{1}{2}} H^l W^l)$$

Key idea 1: weight sharing

Consider this undirected graph:



Calculate update for node in red:



Desirable properties:

- Weight sharing over all locations
- Invariance to permutations
- Linear complexity $O(E)$
- Applicable both in transductive and inductive settings

Update rule:
$$h_i^{(l+1)} = \sigma(h_i^{(l)} W_0^{(l)} + \sum_{j \in N_i} \frac{1}{c_{ij}} h_j^{(l)} W_1^{(l)})$$

N_i : neighbor indices

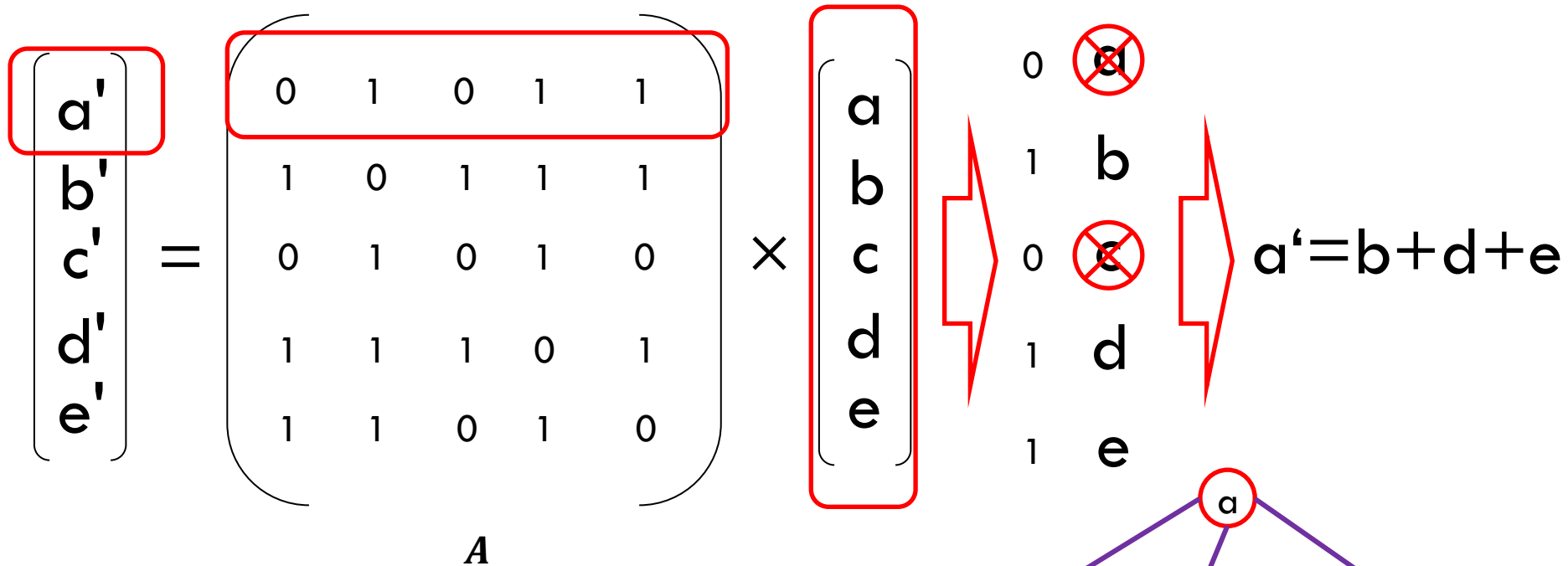
c_{ij} : norm. constant
(fixed/trainable)

Limitations:

- Requires gating mechanism / residual connections for depth
- Only indirect support for edge features

Graph convolutional networks: A concrete model

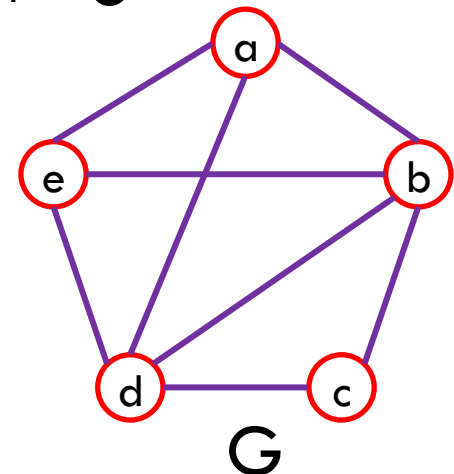
$$H^{(l+1)} = \sigma(\widehat{D}^{-\frac{1}{2}} \widehat{A} \widehat{D}^{-\frac{1}{2}} H^l W^l)$$



Focus on updated a'

Only neighboring nodes of a are retained

Unconnected nodes are masked out



Graph convolutional networks: A concrete model

$$\begin{bmatrix} a' \\ b' \\ c' \\ d' \\ e' \end{bmatrix} = \begin{bmatrix} 1 & 1 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 1 & 1 \end{bmatrix} \times \begin{bmatrix} a \\ b \\ c \\ d \\ e \end{bmatrix}$$

$\hat{A} = A + I_N$

$$a' = a + b + d + e$$

Self-loops are added into adjacency matrix

$$H^{(l+1)} = \sigma(\hat{A}H^l W^l)$$

Central nodes are included in convolution

Key idea 2: Normalizing the Feature Representations

$$H^{(l+1)} = \sigma(\hat{A}H^lW^l) \rightarrow H^{(l+1)} = \sigma(\hat{D}^{-1}\hat{A}H^lW^l)$$

The feature representations can be normalized by node degree by transforming the adjacency matrix $\hat{A}$ by multiplying it with the inverse degree matrix $\hat{D}^{-1}$.

$$\begin{pmatrix} 4 & 0 & 0 & 0 & 0 \\ 0 & 5 & 0 & 0 & 0 \\ 0 & 0 & 3 & 0 & 0 \\ 0 & 0 & 0 & 5 & 0 \\ 0 & 0 & 0 & 0 & 4 \end{pmatrix}^{-1} \begin{pmatrix} 1 & 1 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 1 & 1 \end{pmatrix} \rightarrow \begin{pmatrix} 0.25 & 0.25 & 0 & 0.25 & 0.25 \\ 0.20 & 0.20 & 0.20 & 0.20 & 0.20 \\ 0 & 0.33 & 0.33 & 0.33 & 0 \\ 0.20 & 0.20 & 0.20 & 0.20 & 0.20 \\ 0.25 & 0.25 & 0 & 0.25 & 0.25 \end{pmatrix}$$

Key idea 2: Normalizing the Feature Representations

$$\begin{pmatrix} 4 & 0 & 0 & 0 & 0 \\ 0 & 5 & 0 & 0 & 0 \\ 0 & 0 & 3 & 0 & 0 \\ 0 & 0 & 0 & 5 & 0 \\ 0 & 0 & 0 & 0 & 4 \end{pmatrix} \begin{pmatrix} 1 \\ -\frac{1}{2} \end{pmatrix} \begin{pmatrix} 1 & 1 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 1 & 1 \end{pmatrix} \begin{pmatrix} 4 & 0 & 0 & 0 & 0 \\ 0 & 5 & 0 & 0 & 0 \\ 0 & 0 & 3 & 0 & 0 \\ 0 & 0 & 0 & 5 & 0 \\ 0 & 0 & 0 & 0 & 4 \end{pmatrix} \begin{pmatrix} 1 \\ -\frac{1}{2} \end{pmatrix}$$



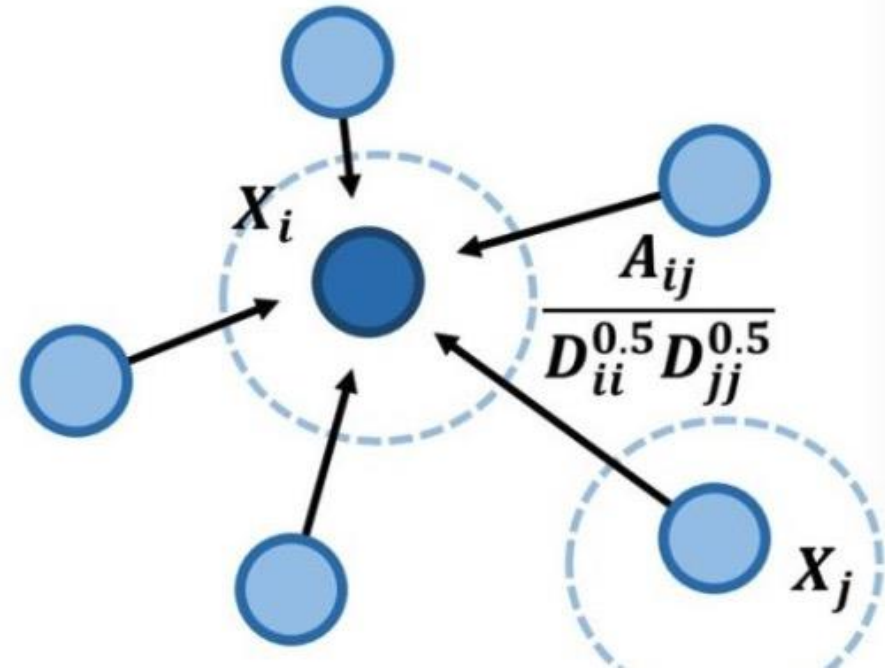
$$\begin{pmatrix} 0.25 & 0.22 & 0 & 0.22 & 0.25 \\ 0.22 & 0.20 & 0.26 & 0.20 & 0.22 \\ 0 & 0.26 & 0.33 & 0.26 & 0 \\ 0.22 & 0.20 & 0.26 & 0.20 & 0.22 \\ 0.25 & 0.22 & 0 & 0.22 & 0.25 \end{pmatrix}$$

A symmetric normalization

$$H^{(l+1)} = \sigma(\hat{D}^{-\frac{1}{2}} \hat{A} \hat{D}^{-\frac{1}{2}} H^l W^l)$$

Key idea 2: Normalizing the Feature Representations

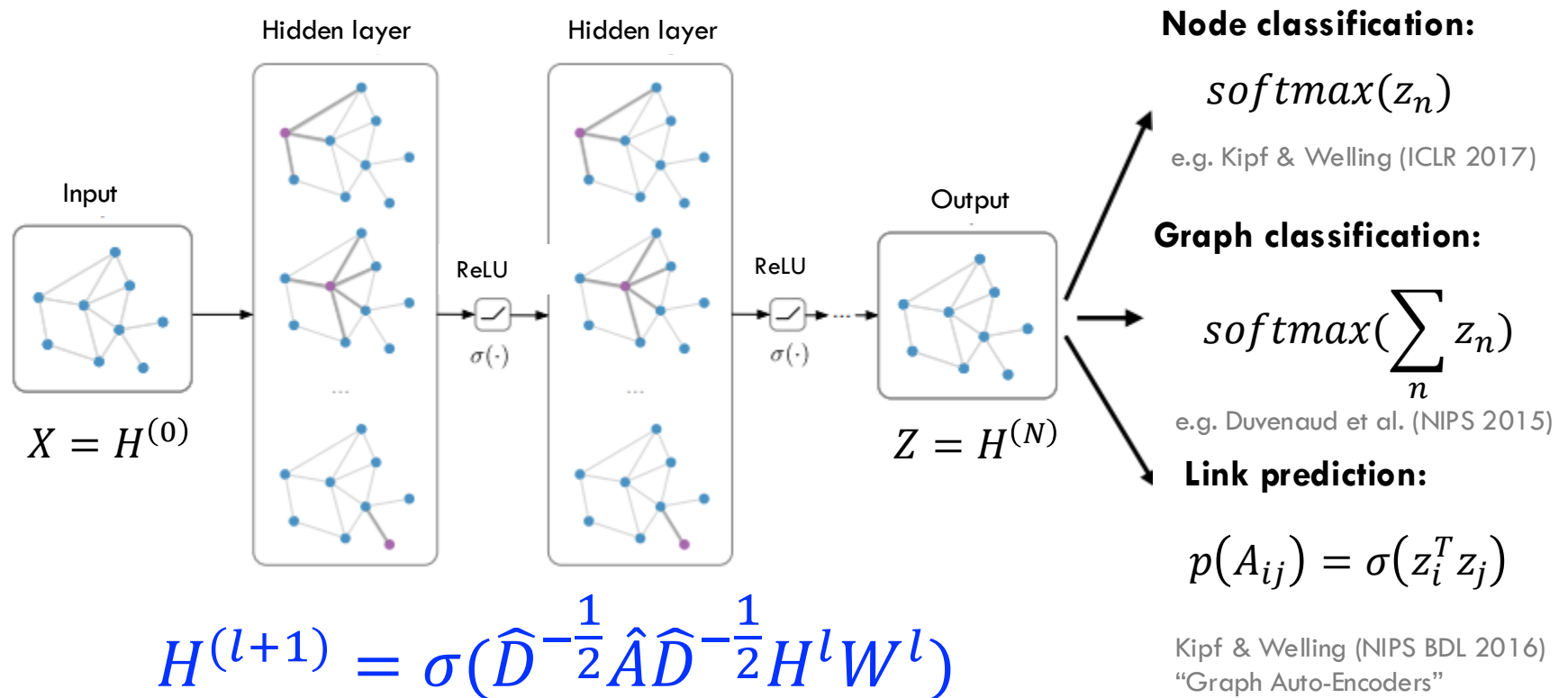
$$\begin{aligned} & (\hat{D}^{-0.5} \hat{A} \hat{D}^{-0.5} H)_i \\ &= (\hat{D}^{-0.5} \hat{A})_i \hat{D}^{-0.5} H \\ &= \left(\sum_k \hat{D}_{ik}^{-0.5} \hat{A}_i \right) \hat{D}^{-0.5} H \\ &= \hat{D}_{ii}^{-0.5} \sum_j \hat{A}_{ij} \sum_k \hat{D}_{ik}^{-0.5} H_j \\ &= \hat{D}_{ii}^{-0.5} \sum_j \hat{A}_{ij} \hat{D}_{jj}^{-0.5} H_j \\ &= \sum_j \frac{1}{\sqrt{\hat{D}_{ii} \hat{D}_{jj}}} \hat{A}_{ij} H_j \end{aligned}$$



Graph convolutional networks: A concrete model

Input: Feature matrix $X \in R^{N \times E}$,
preprocessed adjacency matrix $\hat{A}$

Different prediction tasks



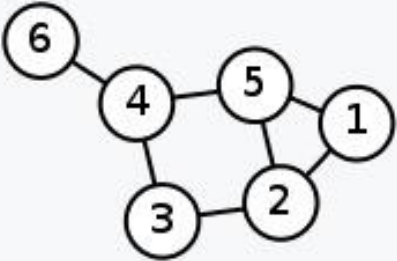
Semi-Supervised Classification with Graph Convolutional Networks (Kipf & Welling)

Spectral approach

Spectral representation of a graph

Graph Laplacian: $L = D - A$

Normalized Graph Laplacian: $L' = I_N - D^{-\frac{1}{2}}AD^{-\frac{1}{2}}$

Labelled graph	Degree matrix	Adjacency matrix	Laplacian matrix
	$\begin{pmatrix} 2 & 0 & 0 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 3 & 0 & 0 \\ 0 & 0 & 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix}$	$\begin{pmatrix} 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{pmatrix}$	$\begin{pmatrix} 2 & -1 & 0 & 0 & -1 & 0 \\ -1 & 3 & -1 & 0 & -1 & 0 \\ 0 & -1 & 2 & -1 & 0 & 0 \\ 0 & 0 & -1 & 3 & -1 & -1 \\ -1 & -1 & 0 & -1 & 3 & 0 \\ 0 & 0 & 0 & -1 & 0 & 1 \end{pmatrix}$

https://en.wikipedia.org/wiki/Laplacian_matrix

D: degree matrix

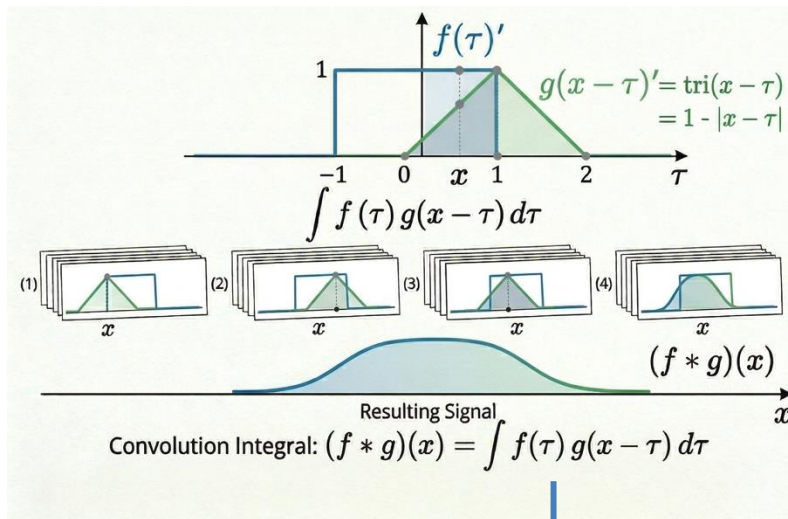
A: adjacency matrix

I_N : identity matrix

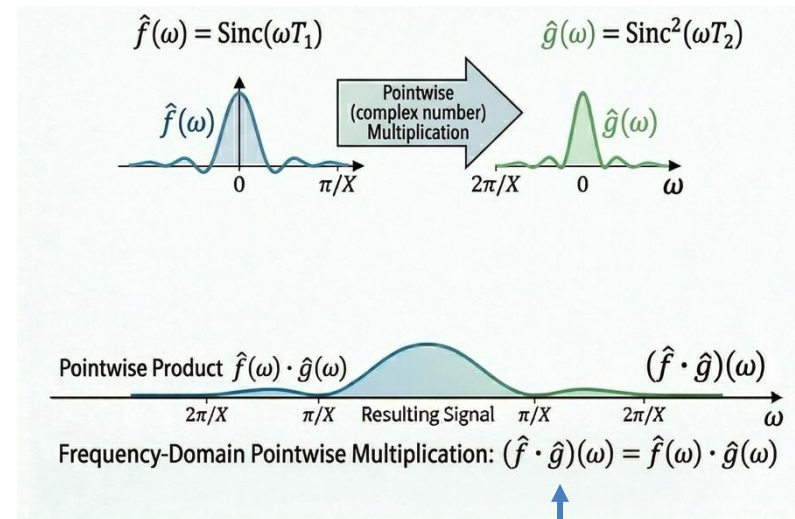
Fourier transform

Convolution theorem: Under suitable conditions, the Fourier (Laplace) transform of a convolution of two signals is the pointwise product of their Fourier (Laplace) transforms.

Time domain convolution



Frequency domain pointwise product



Figures generated with Gemini

Fourier transform

Fourier transform

Convolution theorem: Under suitable conditions, the Fourier (Laplace) transform of a convolution of two signals is the pointwise product of their Fourier (Laplace) transforms.

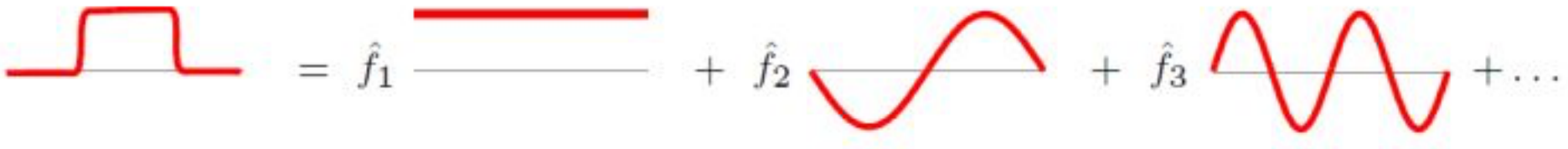
Given a graph f and a convolutional filter (with trainable parameters) h , the convolution can be calculated by

$$f * h = \mathcal{F}^{-1}[\hat{f}(\omega)\hat{h}(\omega)]$$

Fourier transform

Time domain

Frequency domain



The classic Fourier transform (continuous time):

$$\hat{f}(\xi) = \langle f, e^{2\pi i \xi t} \rangle = \int_{\mathbb{R}} f(t) e^{2\pi i \xi t} dt$$

This requires a continuous time variable in the function $f(t)$.

Fourier transform on the graph

The classic Fourier transform

$$\hat{f}(\xi) = \langle f, e^{2\pi i \xi t} \rangle = \int_{\mathbb{R}} f(t) e^{2\pi i \xi t} dt$$

Observation: the expansion of f in terms of the complex exponentials ($e^{2\pi i \xi t}$); i.e. the Fourier transform is the expansion of a signal in terms of the eigenfunctions of 1-d Laplace operator Δ :

$$\underline{-\Delta}(e^{2\pi i \xi t}) = -\frac{\partial^2}{\partial t^2} e^{2\pi i \xi t} = \underline{(2\pi \xi)^2} \underline{e^{2\pi i \xi t}}$$

$$Lu = \lambda u$$

Can use this trick to compute Fourier transform on graph.

Fourier transform on the graph

The classic Fourier transform

$$\hat{f}(\xi) = \langle f, e^{2\pi i \xi t} \rangle = \int_{\mathbb{R}} f(t) e^{2\pi i \xi t} dt$$

Graph Fourier transform $\hat{f}$: of any $f \in \mathbb{R}^N$, of all vertices of G , expansion of f :

$$\hat{f}(\lambda_l) = \langle f, u_l \rangle = \sum_i^N f(i) u_l^*(i) \qquad \hat{f} = U^T f$$

Similarly,

$$\hat{h}(\lambda_l) = \langle h, u_l \rangle = \sum_i^N h(i) u_l^*(i) \qquad \hat{h} = U^T h$$

The inverse graph Fourier transform is then given by:

$$f(i) = \sum_{l=0}^{N-1} \hat{f}(\lambda_l) u_l(i) \qquad f = U \hat{f}$$

Graph convolution

Convolution theorem: Under suitable conditions, the Fourier (Laplace) transform of a convolution of two signals is the pointwise product of their Fourier (Laplace) transforms.

Given a graph f and a convolutional filter (with trainable parameters) h , the convolution can be calculated by

$$f * h = \mathcal{F}^{-1}[\hat{f}(\omega)\hat{h}(\omega)]$$

$$\underbrace{\hat{f} = U^T f \quad \hat{h} = U^T h \quad f = U \hat{f}}_{\text{Fourier (Laplace) transform}}$$

$$(f * h)_G = U((U^T f) \odot (U^T h))$$

Graph convolutional network

$$(f * h)_G = U((U^T f) \odot (U^T h))$$

Version 1.0 (Spectral Networks and Locally Connected Networks on Graphs)

$$y = \sigma(U g_{\theta}(\Lambda) U^T x)$$

$$g_{\theta}(\Lambda) = \begin{bmatrix} \theta_1 & 0 & 0 \\ 0 & \ddots & 0 \\ 0 & 0 & \theta_n \end{bmatrix}$$

- $U^T f$ and $U^T h$ are vectors of size $n \times 1$, $g_{\theta}(\Lambda)$ is $n \times n$
- Calculate the multiplication between U , $g_{\theta}(\Lambda)$, and U^T in each feedforward.
- #param = n
- No spatial localization

Graph convolutional network

- No eigen decomposition
- Spatial localization
- #param = K

$$(f * h)_G = U((U^T f) \odot (U^T h))$$

Version 2.0 (Convolutional Neural Networks on Graphs with Fast Localized Spectral Filtering)

$$y = \sigma(U g_{\theta}(\Lambda) U^T x)$$

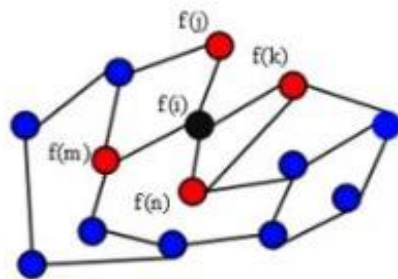
$$g_{\theta}(\Lambda) = \begin{bmatrix} \sum_{j=0}^K \alpha_j \lambda_1^j & 0 & 0 \\ 0 & \ddots & 0 \\ 0 & 0 & \sum_{j=0}^K \alpha_j \lambda_n^j \end{bmatrix} = \sum_{j=0}^K \alpha_j \Lambda^j \longrightarrow \begin{aligned} & U g_{\theta}(\Lambda) U^T \\ &= U \sum_{j=0}^K \alpha_j \Lambda^j U^T \\ &= \sum_{j=0}^K \alpha_j L^j \end{aligned}$$

Graph convolutional network

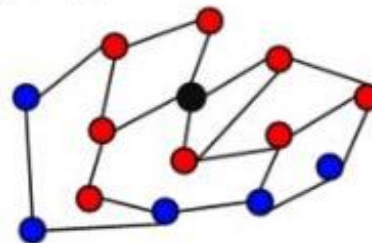
$$y = \sigma(U g_{\theta}(\Lambda) U^T x) = \sigma \left(\sum_{j=0}^K \alpha_j L^j x \right)$$

- There are only K parameters to train, and usually $K \ll n$. The complexity has been decreased.
- No need to conduct eigen decomposition. Explicitly depend on the Laplacian matrix L .
- Spatial localization. In particular, K denotes the receptive field.

$K = 1$



$K = 2$

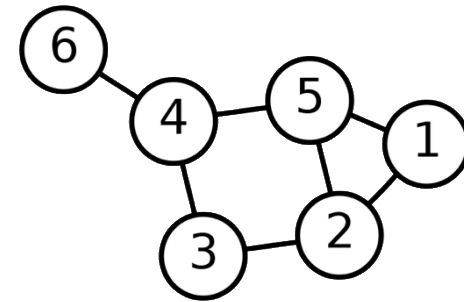


Graph convolutional network

Local Connectivity + Parameter Sharing

$K=0$, the convolution filter is

$$\begin{bmatrix} \alpha_0 & 0 & 0 & 0 & 0 & 0 \\ 0 & \alpha_0 & 0 & 0 & 0 & 0 \\ 0 & 0 & \alpha_0 & 0 & 0 & 0 \\ 0 & 0 & 0 & \alpha_0 & 0 & 0 \\ 0 & 0 & 0 & 0 & \alpha_0 & 0 \\ 0 & 0 & 0 & 0 & 0 & \alpha_0 \end{bmatrix}$$



$K=1$, the convolution filter is

$$\begin{bmatrix} \alpha_0 + 2\alpha_1 & -\alpha_1 & 0 & 0 & -\alpha_1 & 0 \\ -\alpha_1 & \alpha_0 + 3\alpha_1 & -\alpha_1 & 0 & -\alpha_1 & 0 \\ 0 & -\alpha_1 & \alpha_0 + 2\alpha_1 & -\alpha_1 & 0 & 0 \\ 0 & 0 & -\alpha_1 & \alpha_0 + 3\alpha_1 & -\alpha_1 & -\alpha_1 \\ -\alpha_1 & -\alpha_1 & 0 & -\alpha_1 & \alpha_0 + 3\alpha_1 & 0 \\ 0 & 0 & 0 & -\alpha_1 & 0 & \alpha_0 + \alpha_1 \end{bmatrix}$$

$K=2$, the convolution filter is

$$\begin{bmatrix} \alpha_0 + 2\alpha_1 + 6\alpha_2 & -\alpha_1 - 4\alpha_2 & \alpha_2 & \alpha_2 & -\alpha_1 - 4\alpha_2 & 0 \\ -\alpha_1 - 4\alpha_2 & \alpha_0 + 3\alpha_1 + 12\alpha_2 & -\alpha_1 - 5\alpha_2 & 2\alpha_2 & -\alpha_1 - 5\alpha_2 & 0 \\ \alpha_2 & -\alpha_1 - 5\alpha_2 & \alpha_0 + 2\alpha_1 + 6\alpha_2 & -\alpha_1 - 5\alpha_2 & 2\alpha_2 & \alpha_2 \\ \alpha_2 & 2\alpha_2 & -\alpha_1 - 5\alpha_2 & \alpha_0 + 3\alpha_1 + 12\alpha_2 & -\alpha_1 - 6\alpha_2 & -\alpha_1 - 4\alpha_2 \\ -\alpha_1 - 4\alpha_2 & -\alpha_1 - 5\alpha_2 & 2\alpha_2 & -\alpha_1 - 6\alpha_2 & \alpha_0 + 3\alpha_1 + 12\alpha_2 & \alpha_2 \\ 0 & 0 & \alpha_2 & -\alpha_1 - 4\alpha_2 & \alpha_2 & \alpha_0 + \alpha_1 + 2\alpha_2 \end{bmatrix}$$

Graph convolutional network

A Recursive formulation for fast filtering

Recall:

$$y = \sigma(U g_{\theta}(\Lambda) U^T x)$$

With Chebyshev expansion, we have

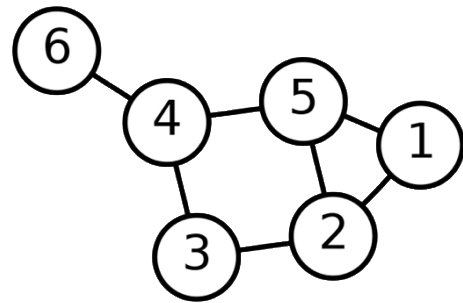
$$g_{\theta'} \approx \sum_{j=0}^K \theta_j' T_j(\tilde{\Lambda}) \quad \text{where} \quad \tilde{\Lambda} = \frac{2\Lambda}{\lambda_{\max}} - I_N$$

Rescaled value
between $[-1, 1]$

Chebyshev polynomials: $T_j(x) = 2xT_{j-1}(x) - T_{j-2}(x)$,
with $T_0(x) = 1$ and $T_1(x) = x$

$$\begin{aligned} y &= \sigma(U g_{\theta}(\Lambda) U^T x) = \sigma\left(U \sum_{j=0}^K \theta_j' T_j(\tilde{\Lambda}) U^T x\right) = \sigma\left(\sum_{j=0}^K \theta_j' T_j(\mathbf{U} \tilde{\Lambda} \mathbf{U}^T) x\right) \\ &= \sigma\left(\sum_{j=0}^K \theta_j' T_j(\tilde{L}) x\right) \end{aligned}$$

Graph convolutional network



$$L = I - D^{-\frac{1}{2}} A D^{-\frac{1}{2}}$$

K=0, the convolution filter is

$$\begin{bmatrix} \beta_0 & 0 & 0 & 0 & 0 & 0 \\ 0 & \beta_0 & 0 & 0 & 0 & 0 \\ 0 & 0 & \beta_0 & 0 & 0 & 0 \\ 0 & 0 & 0 & \beta_0 & 0 & 0 \\ 0 & 0 & 0 & 0 & \beta_0 & 0 \\ 0 & 0 & 0 & 0 & 0 & \beta_0 \end{bmatrix}$$

K=1, the convolution filter is

$$\begin{bmatrix} \beta_0 + 0.07\beta_1 & -0.44\beta_1 & 0 & 0 & -0.44\beta_1 & 0 \\ -0.44\beta_1 & \beta_0 + 0.07\beta_1 & -0.44\beta_1 & 0 & -0.36\beta_1 & 0 \\ 0 & -0.44\beta_1 & \beta_0 + 0.07\beta_1 & -0.44\beta_1 & 0 & 0 \\ 0 & 0 & -0.44\beta_1 & \beta_0 + 0.07\beta_1 & -0.36\beta_1 & -0.62\beta_1 \\ -0.36\beta_1 & -0.36\beta_1 & 0 & -0.36\beta_1 & \beta_0 + 0.07\beta_1 & 0 \\ 0 & 0 & 0 & -0.62\alpha_1 & 0 & \beta_0 + 0.07\beta_1 \end{bmatrix}$$

- Considering the diagonal entry, the coefficient before β_1 is small.
- β_1 is to control the properties of the one-hop neighbor.

Thank you!